



Python Programming - 2301CS404

Lab - 12

Roll No : 418

Name : Bhavik A. Parmar

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [16]: # ZeroDivisionError
try:
    a = int(input("Enter Number1 :"))
    b = int(input("Enter Number2 :"))
    c = a / b
    print("Ans : ", c)
except ZeroDivisionError as err:
    print("ZeroDivisionError : ", err)
print("End OF Program...")

# ValueError
try:
    a = int(input("Enter Number1 :"))
    b = int(input("Enter Number2 :"))
    c = a / b
    print("Ans : ", c)
except ValueError as err:
    print("ValueError : ", err)
print("End OF Program...")
```

```
# TypeError
try:
    print(10 + "5")
except TypeError as err:
    print("TypeError : ", err)
print("End OF Program...")
```

ZeroDivisionError : division by zero

End OF Program...

ValueError : invalid literal for int() with base 10: 'abc'

End OF Program...

TypeError : unsupported operand type(s) for +: 'int' and 'str'

End OF Program...

02) WAP to handle following exceptions:

1. IndexError

2. KeyError

```
In [22]: # IndexError
l1 = [1, 2, 3, 4, 5]
try:
    idxL = int(input("Enter Index :"))
    print(l1[idxL])
except IndexError as err:
    print("IndexError : ", err)
print("End OF Program...")

# KeyError
d1 = {1 : "One", 2 : "Two", 3 : "Three"}
try:
    idxD = int(input("Enter Index :"))
    print(d1[idxD])
except KeyError as err:
    print("KeyError : ", err)
print("End OF Program...")
```

IndexError : list index out of range

End OF Program...

KeyError : 10

End OF Program...

03) WAP to handle following exceptions:

1. FileNotFoundError

2. ModuleNotFoundError

```
In [18]: # FileNotFoundError
try:
    fp = open("Demo.txt", 'r')
    data = fp.read()
    print(data)
    fp.close()
except FileNotFoundError as err:
    print("FileNotFoundError : ", err)
print("End OF Program")
```

```
# ModuleNotFoundError
try:
    import DevoModule
except ModuleNotFoundError as err:
    print("ModuleNotFoundError : ", err)
print("End OF Program")
```

FileNotFoundError : [Errno 2] No such file or directory: 'Demo.txt'
 End OF Program
 ModuleNotFoundError : No module named 'DevoModule'
 End OF Program

04) WAP that handles all type of exceptions in a single except block in standard error message format.

```
In [32]: l1 = [1, 2, 3, 4, 5]
d1 = {1 : "One", 2 : "Two", 3 : "Three"}
try:
    a = int(input("Enter Number1 :"))
    b = int(input("Enter Number2 :"))
    c = a / b

    idxL = int(input("Enter Index OF List:"))
    print(l1[idxL])

    idxD = int(input("Enter Index :"))
    print(d1[idxD])

    print(10 + "5")
    print(x)
    fp = open("Demo.txt", 'r')

except Exception as err:
    print("Error :", err)
print("End OF Program...")
```

Error : division by zero
 End OF Program...

05) WAP to demonstrate else and finally block.

```
In [33]: try:
    a = int(input("Enter Number1 :"))
    b = int(input("Enter Number2 :"))
    c = a / b
except Exception as err:
    print("Error : ", err)
else:
    print("Ans : ", c)
finally:
    print("End OF Program")
```

Error : division by zero
 End OF Program

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

```
In [41]: def divide(a, b):
    try:
        c = a // b
    except ZeroDivisionError as err:
        print("ZeroDivisionError :", err)
    else:
        print("Sucessfully")
        return c
    finally:
        print("End OF Program")

a = int(input("Enter Number1 :"))
b = int(input("Enter Number2 :"))
ans = divide(a, b)
print("Ans : ", ans)
```

ZeroDivisionError : integer division or modulo by zero
 End OF Program
 Ans : None

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

```
In [18]: bill = 0
    try:
        prices = list(map(int, input("Enter Prices OF Items :").split(" ")))
        for i in prices:
            if i < 0:
                raise ValueError("Prices Can't be Negative.")
    except ValueError as err:
        print("Error :", err)
    else:
        bill = sum(prices)
        print("Total Bill Amount:", bill)
    finally:
        print("End OF Program.")
```

Error : Prices Can't be Negative.
 End OF Program.

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

```
In [17]: p = input("Enter Grades : ").split(',')
    try:
        g = [int(grade) for grade in p]
```

```
except ValueError as err:
    print("Error : ", err)
finally:
    print("End OF Program...")
```

Error : invalid literal for int() with base 10: 'A'
End OF Program...

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

```
In [23]: sum = 0
avg = 0
try:
    l1 = []
    for i in range(1, 6):
        m = int(input("Enter Your Marks :"))
        assert 0 < m < 100, 'Marks Should be between 0 to 100'
        sum += m
        avg = sum / 5
except AssertionError as err:
    print("Error :", err)
else:
    print("Avg Marks : ", avg)
finally:
    print("End OF Program...")
```

Error : Marks Should be between 0 to 100
End OF Program...

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the message "Correct" otherwise.

```
In [5]: class WeakPasswordError(Exception):
    pass
try:
    p = input("Enter Your Password :")
    if len(p) < 8 or not any(i.isdigit() for i in p):
        raise WeakPasswordError("Password must have at least 8 characters and on
except WeakPasswordError as err:
```

```

    print("Error :", err)
else:
    print("Sucessfully Set Your Password, Correct")
finally:
    print("End OF Program...")

```

Error : Password must have at least 8 characters and one digit.
End OF Program...

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```

In [8]: import math
class NegativeNumberError(Exception):
    pass

try:
    x = int(input("Enter Number :"))
    if x < 0:
        raise NegativeNumberError("Cannot Calculate the Square root of a Negative")
    ans = math.sqrt(x)
except NegativeNumberError as err:
    print("Error :", err)
else:
    print("Square Root OF Number :", ans)
finally:
    print("End OF Program...")

```

Error : Cannot Calculate the Square root of a Negative Number.
End OF Program...

In []: